VERIQTA

Production Maintenance & Zero-Downtime Changes



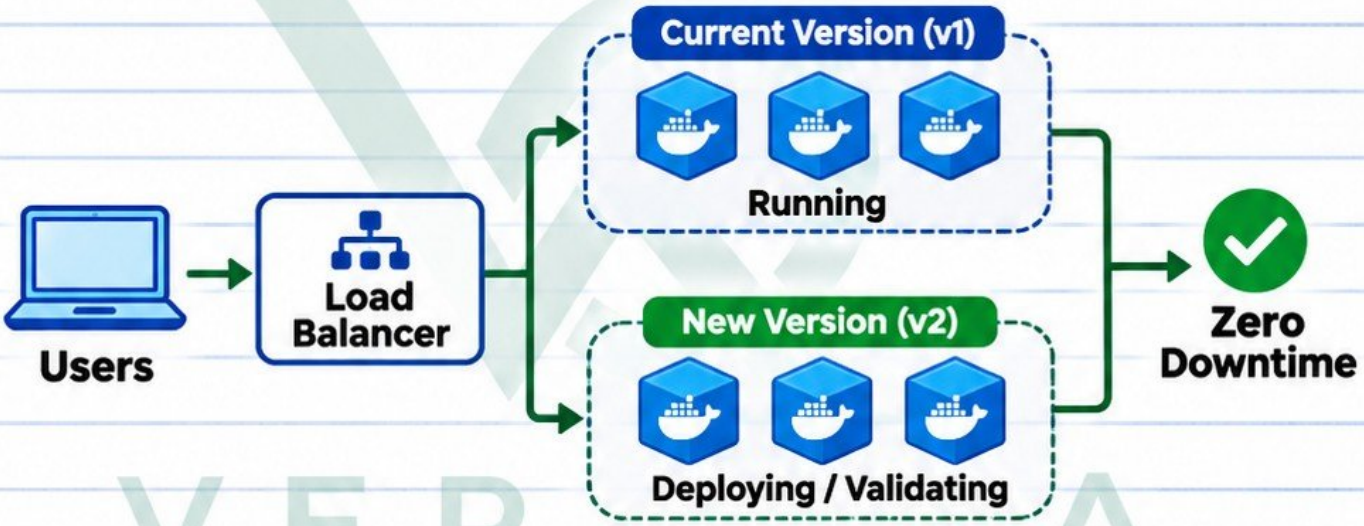
PLAN SAFELY | DEPLOY RELIABLY | MAINTAIN CONFIDENCE | KEEP SERVICES ONLINE

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

Production Maintenance & Zero-Downtime Changes

Practical Strategies to Keep Applications Running While You Deploy, Update, and Fix



- Safe Deployment Strategies
- Rolling and Blue–Green Updates
- Health Checks and Rollback
- Production Maintenance Tasks
- Real-World Examples and Commands

You Will Learn:

- How to update without downtime
- How to keep services healthy
- How to undo a bad change
- How to plan and perform production maintenance

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Production Maintenance Engineering Fundamentals

Core concepts for keeping production systems reliable, available, and safe while making changes.

What Is Production Maintenance?

Planned and controlled work to keep production systems healthy, secure, and continuously available.

Why It Matters

- Keeps services reliable
- Reduces unexpected outages
- Improves security and performance
- Allows safe, controlled changes

Maintenance Categories

- **Preventive** – Regular, planned work (e.g., updates, patching).
- **Corrective** – Fixes issues after a problem is found.
- **Adaptive** – Adjusts to changes (e.g., new requirements, scaling).
- **Perfective** – Improves performance, reliability, or usability.

Operational Risk

The chance that a change or activity can cause a service outage, data loss, or degraded performance.

Risk Mitigation

- Plan and test changes
- Use rollback options
- Monitor before, during, and after
- Start with lower-risk environments

Availability Objectives

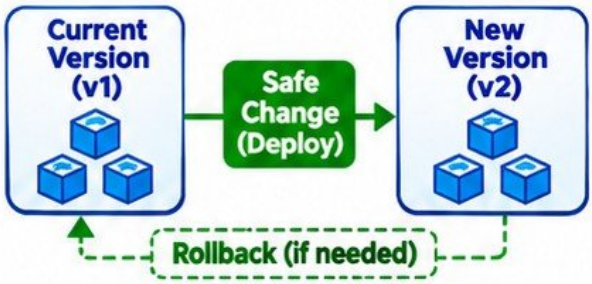
- Uptime target (e.g., 99.9%).
- Defines how much downtime is acceptable.
- Guides maintenance planning and change decisions.

Maintenance Windows

- A scheduled period for planned maintenance or changes.
- Communicated in advance.
- Used for higher-risk or disruptive work.

Engineering Principles for Safe Changes

- **Plan:** Define scope, impact, and rollback.
- **Test:** Validate the change in a non-production environment.
- **Deploy Gradually:** Use strategies like rolling or blue-green.
- **Monitor:** Watch metrics, logs, and health checks.
- **Rollback:** Quickly restore the previous stable version if needed.



Understanding Zero Downtime and Availability Guarantees

Key concepts, targets, and trade-offs for keeping services continuously available in production.

What Is Zero Downtime?

A deployment or change that does not cause a service outage or user disruption.

What Is Availability?

The percentage of time a service is up and accessible to users over a period.

Service-Level Objectives (SLOs)

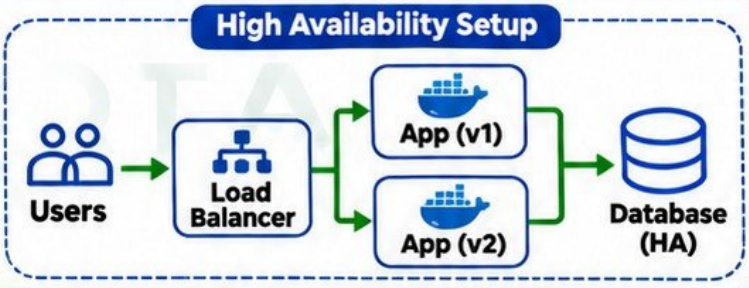
- A measurable target for service performance.
- Example: 99.9% monthly availability.
- Helps the team balance reliability, cost, and speed of change.

Error Budgets

- The allowed amount of unavailability in a given period.
- Example: For 99.9% availability, the budget is 0.1% ≈ 43 minutes per month.
- If you exceed it, focus on stability instead of new changes.

Redundancy Requirements

- [x] Run multiple instances (no single point of failure).
- [x] Use a load balancer to spread traffic.
- [x] Deploy across multiple zones or nodes.
- [x] Add automated health checks and failover.



Limitations of Availability Guarantees

- It is not a guarantee of zero failures.
- Outages can still happen (e.g., network, cloud, dependencies).
- Higher availability increases cost and complexity.
- Some risks (e.g., regional outages) are harder to eliminate completely.

Key Takeaways

- [x] Zero downtime is an engineering goal, not a simple setting.
- [x] Use SLOs to define a realistic target.
- [x] Track error budgets to guide change speed.
- [x] Design with redundancy and automated failover.
- [x] Understand the trade-offs: higher availability costs more and adds complexity.

Production Change Risk Assessment

A structured way to evaluate the risk, impact, and safety of changes before they go to production.

What Is a Change Risk Assessment?



A pre-change evaluation that identifies possible risks, impact, and mitigation before making a production change.

Blast Radius



The scope of impact if the change goes wrong (e.g., one service, multiple services, or the entire platform).

Dependency Mapping



- Identify all components that depend on, or are affected by, the change.
- Helps you understand upstream and downstream impact.

Failure Domains



- A logical boundary that limits how far a failure can spread (e.g., a service, cluster, or availability zone).
- Design changes to contain failures within a domain.

Risk Classification

Level	Definition	Example
Low	Minimal impact and easy to rollback.	Config change, non-critical update.
Medium	Noticeable impact but controlled.	Service update, resource change.
High	Major impact, higher chance of outage.	Database change, architecture change.

Change Impact Analysis



- ✅ What will change?
- ✅ Which users or services are affected?
- ✅ Expected downtime (if any)?
- ✅ How will you monitor it?
- ✅ What is the rollback plan?

Go / No-Go Criteria

✅ **Go:** Risk is accepted, mitigations are in place, rollback is ready, and the change meets objectives.

❌ **No-Go:** Risk is too high, impact is unclear, dependencies are not ready, or rollback is not available.

Key Takeaways

- ✅ Assess risk before every production change.
- ✅ Understand the blast radius and dependencies.
- ✅ Use failure domains to limit impact.
- ✅ Classify the risk and prepare mitigations.
- ✅ Proceed only when go criteria are met.

Designing Safe Production Maintenance Plans

A practical approach to plan, execute, and verify production maintenance with minimal risk and no unnecessary downtime.

What Is a Maintenance Plan?

A documented, step-by-step plan to make changes, keep services stable, and quickly recover if needed.

Maintenance Runbook

- Clear, repeatable steps.
- Required commands and tools.
- Success and rollback steps.
- Who does what and when.

Prerequisites

- Approved change request.
- Required access and tools.
- Backups or snapshots taken.
- Rollback plan prepared.

Preflight Checks

- Verify system health.
- Check resource capacity (CPU, memory, disk).
- Confirm dependencies are up.
- Ensure backups are valid.

Stakeholder Communication

- Announce the change (what, why, when, and expected impact).
- Notify affected teams and users.
- Share a status update (start, progress, completion).

Execution Sequencing

1. Prepare → 2. Back Up → 3. Apply Change → 4. Verify

Rollback Criteria

- Service becomes unhealthy or error rate increases.
- Performance degrades beyond the agreed SLO.
- Unexpected or wider impact is detected.
- The change cannot be validated within the expected time.

Verification (Post-Change)

- Confirm all services are healthy.
- Validate key user flows and APIs.
- Check logs, metrics, and alerts.
- Ensure performance is within expected levels.

Key Takeaways

- A good plan reduces risk and speeds up recovery.
- Always verify before and after the change.
- Communication and a tested rollback make the change safe.

High Availability Architecture for Maintenance

Design a resilient system that stays available during maintenance, failures, or updates.

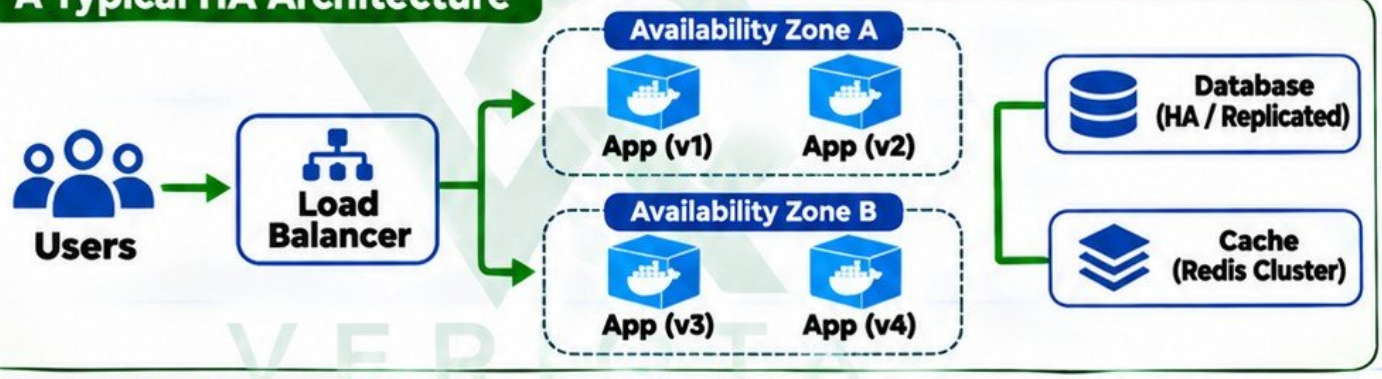
What Is High Availability (HA)?

An architecture that keeps services running with minimal or no downtime, even when parts fail or are being maintained.

Why It Matters

- Keep services available during changes.
- Handle unexpected failures.
- Improve reliability and trust.
- Support zero-downtime updates.

A Typical HA Architecture



Key Building Blocks

- **Redundant Instances** – Run multiple copies of your application.
- **Availability Zones** – Spread instances across isolated locations (e.g., AZ-A, AZ-B).
- **Load Balancer** – Distributes traffic and keeps healthy instances in service.
- **Health Checks** – Automatic tests to detect and remove unhealthy instances.

Quorum Requirements

- A minimum number of nodes that must agree (e.g., 2 of 3) for the system to stay consistent.
- Common in databases (e.g., etcd, Consul, Kafka) and clustered services.

Eliminating Single Points of Failure

- Avoid only one instance, one database, or one network path.
- Use multiple instances, zones, and replicas.
- Remove dependency on a single component.
- Test failover regularly.

Maintenance Benefits of HA

- Take nodes offline one at a time.
- Update or patch without user impact.
- Automatically route traffic to healthy nodes.
- Recover quickly from unexpected failures.

Rolling Updates Without Service Interruption

Update a live production service safely, gradually, and reliably while keeping it available to users.

What Is a Rolling Update?



A deployment strategy that gradually replaces old instances with new ones, so the service stays available.

Batch Sizing



- Update a small number of instances at a time (a batch).
- Controls risk and limits user impact.
- Typical examples: 10%, 25%, or 1 instance at a time.

Deployment Surge Capacity



- Temporarily run extra instances during the rollout.
- Keeps enough capacity to handle traffic.
- Example: 4 running + 1 surge (25% surge).

Readiness Gates



- Checks that a new instance is healthy and ready before it receives traffic.
- Examples: readiness probe, health endpoint (/health), or smoke test.

Connection Draining



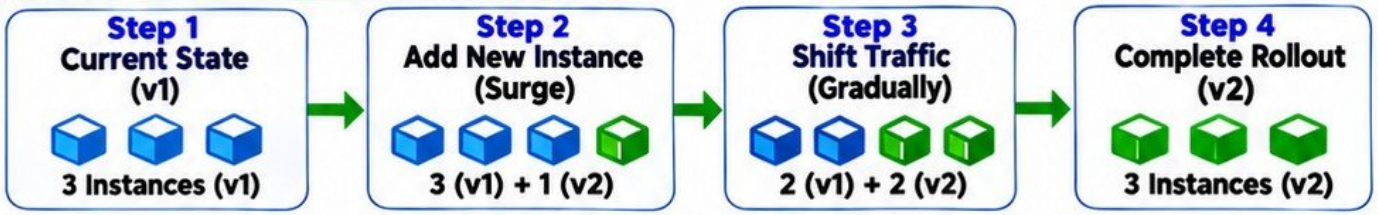
- Stop sending new traffic to an instance before it is removed.
- Let existing requests complete (e.g., 30–120 seconds).
- Prevents dropped or interrupted requests.

Termination Grace Periods



- A timeout that gives the instance time to finish in-flight work before it is terminated.
- Example: 30s, 60s, or 120s depending on the workload (e.g., web request, job, or queue).

Rolling Update Flow (Example)



Rollout Failure Handling



- Detect issues via health checks, error rates, and key metrics.
- Automatically pause the rollout.
- Rollback to the last stable version if thresholds ae breached.
- Investigate, fix, and re-run the rollout.

Key Takeaways

- ☑ Use small, controlled batches to reduce risk.
- ☑ Keep surge capacity to maintain performance.
- ☑ Validate each instance with readiness gates.
- ☑ Drain connections and use a proper grace period.
- ☑ Have a clear rollback plan if the rollout fails.

Blue-Green Deployment and Environment Switching

A zero-downtime release strategy that runs two parallel environments and switches user traffic safely.

What Is Blue-Green Deployment?



A release approach that runs two identical environments (Blue = current, Green = new) and switches traffic between them.

Parallel Environments

- **Blue (Live)**
- **Green (New)**

- Run two complete, isolated environments.
- Test the new version on Green without user impact.
- Keep both environments ready for quick traffic switching.

Traffic Cutover (How It Works)



Configuration Parity



- Keep both environments identical (code, config, infra).
- Use the same environment variables and secrets.
- Avoid environment-specific drifts that cause unexpected behavior.

Session Continuity



- Ensure users stay logged in during and after the switch.
- Use shared session storage (e.g., Redis, database, or token-based sessions).
- Avoid in-memory sessions tied to a single instance.

Database Compatibility



- Apply backward-compatible database changes.
- Use expand–migrate–contract (e.g., add new columns, then migrate data, then remove old).
- Ensure both Blue and Green can work with the same database version.

Blue-Green Deployment Flow

1. **Prepare** — Build and deploy the new version to Green.
2. **Validate** — Test Green (health, smoke, key flows).
3. **Switch** — Update the load balancer to route traffic to Green.
4. **Monitor** — Watch metrics, errors, and user behavior.
5. **Rollback (If Needed)** — Switch traffic back to Blue quickly and safely.

Rollback Constraints



- Rollback is fast, but only if both environments remain compatible.
- Database changes may limit or block rollback.
- Incompatible schema changes can make reversal complex.
- Keep Blue available until Green is fully stable.

Key Takeaways

- [x] Run two parallel, identical environments.
- [x] Validate the new version before switching.
- [x] Ensure session and data compatibility.
- [x] Switch traffic using a load balancer or router.
- [x] Keep a tested and realistic rollback plan.

Canary Releases and Progressive Traffic Shifting

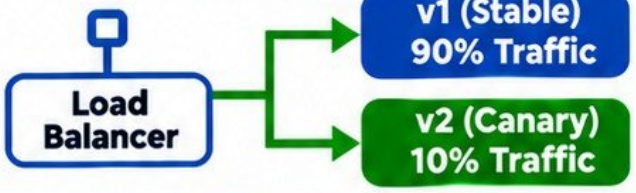
A low-risk release strategy that exposes a new version to a small number of users, validates it with real metrics, and then gradually increases traffic.

What Is a Canary Release?



A deployment approach that releases a new version to a small portion of users first, monitors it, and then gradually rolls it out.

Weighted Routing



Deployment Cohorts



- Release to a small, targeted group (e.g., 1%, 5%, 10%).
- Use real users and real traffic.
- Expand to a larger cohort only when metrics are healthy.

Automated Analysis



- Continuously compare key metrics between old and new versions.
- Use automation to detect regressions, errors, or performance issues.

Metric Thresholds



- Define clear, measurable SLOs/thresholds.
- Examples: error rate < 1%, latency < 300 ms, and success rate > 99.9%.
- Use monitoring and alerts to enforce these thresholds.

Traffic Promotion (Progressive Rollout)



- Gradually increase traffic to the new version (e.g., 10% → 25% → 50% → 100%).
- Confirm metrics remain healthy at each step.
- Complete the rollout when it meets all success criteria.

Canary Release Flow (Example)

1. Deploy — Release v2 to a small % (e.g., 10%).

2. Monitor — Track key metrics (errors, latency, throughput).

3. Analyze — Compare v1 vs v2 against thresholds.

4. Promote — Increase traffic (e.g., 25% → 50% → 100%).

5. Complete or Abort — Finish rollout or rollback if thresholds are breached.

Abort Conditions



- Error rate or failure rate exceeds the limit.
- Latency or resource usage spikes.
- User experience degrades (e.g., high 5xx errors).
- Detected regression or unexpected behavior.

Key Takeaways

- ✓ Use canary releases to reduce deployment risk.
- ✓ Route traffic with weights and clear thresholds.
- ✓ Automate metric analysis and decision-making.
- ✓ Promote gradually and roll back quickly if needed.

Kubernetes Zero-Downtime Maintenance

Run safe maintenance and changes on Kubernetes without service interruption, data loss, or user impact.

What It Covers

Practical Kubernetes techniques to keep applications available during updates, maintenance, and infrastructure changes.

Deployment Strategies

- RollingUpdate – Gradually replace pods (default, zero-downtime).
- Recreate – Stop then start (causes downtime).
- Blue-Green – Switch between two environments.
- Canary – Release to a small percentage first.

Readiness Probes

- Checks if a pod is ready to receive traffic.
- Keeps new pods out of service until healthy.
- Prevent traffic from going to unready or initializing pods.

PodDisruptionBudgets (PDB)

- Limits the number of pods that can be disrupted at once.
- Keeps a minimum number of pods available during maintenance.
- Example: minAvailable: 2 or maxUnavailable: 1.

Graceful Termination

- Allows pods to finish in-flight requests before stopping.
- Controlled by terminationGracePeriodSeconds (e.g., 30s–120s).
- Use preStop hooks for cleanup (e.g., close connections, flush jobs).

Topology Spread

- Spreads pods across nodes, zones, or regions.
- Improves availability and reduces blast radius.
- Example: spread across multiple availability zones (AZs).

Node Drain Operation (Safe Node Maintenance)

1. **Cordon** – Mark the node unschedulable (no new pods).
2. **Drain** – Evict running pods safely (respects PDB).
3. **Maintain** – Perform the node update (e.g., patch, reboot).
4. **Uncordon** – Mark the node schedulable again.

Key Commands (Examples)

```
$ kubectl rollout status deployment/<app>
$ kubectl get pdb
$ kubectl drain <node> --ignore-daemonsets
$ kubectl uncordon <node>
```

Key Takeaways

- [x] Use the right deployment strategy for zero downtime.
- [x] Always configure readiness probes.
- [x] Apply PodDisruptionBudgets to maintain availability.
- [x] Use graceful termination and preStop hooks.
- [x] Spread pods across nodes/zones for higher resilience.
- [x] Drain and maintain nodes safely using cordon and drain.

Kubernetes Node Upgrades and Cluster Maintenance

Safely upgrade and maintain Kubernetes nodes and clusters without causing service interruption.

What It Covers



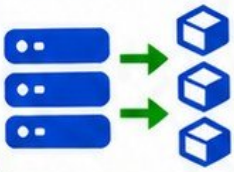
A practical approach to upgrading nodes and maintaining a Kubernetes cluster while keeping workloads available and stable.

Node Cordoning



- Mark a node as unschedulable (cordon) so no new pods are placed.
- Prevents new workloads during maintenance or upgrade.
- Keeps existing pods running until they are drained.

Node Draining



- Safely evict pods from a node (drain).
- Respects PodDisruptionBudgets (PDBs) and graceful termination.
- Ensures workloads are moved to healthy nodes first.

Replacement Sequencing



- Upgrade or replace nodes in a controlled, one-at-a-time or batched sequence.
- Maintain desired capacity (e.g., keep N-1 nodes available).
- Avoids large-scale disruptions and capacity drops.

Cluster Autoscaler Interactions



- Automatically adds nodes if capacity is low during the upgrade.
- May temporarily scale out when pods are rescheduled.
- Ensure autoscaler settings (e.g., min/max nodes) are correct before maintenance.

Workload Rescheduling



- Moves evicted pods to other healthy nodes automatically.
- Relies on the scheduler and resource availability.
- Consider resource requests/limits and topology constraints (e.g., zones, nodes).

Upgrade Validation



- Confirm all nodes are on the target version.
- Verify pods are healthy and services are available.
- Check key metrics (e.g., errors, latency, restarts) for stability.

Node Upgrade Flow (Example)

1. **Cordon** — Mark node unschedulable.
2. **Drain** — Evict pods safely.
3. **Upgrade / Replace** — Upgrade or replace node.
4. **Validate** — Confirm node and workloads are healthy.

Key Commands (Examples)

```
$ kubectl get nodes
$ kubectl cordon <node>
$ kubectl drain <node> --ignore-daemonsets
$ kubectl uncordon <node>
$ kubectl get nodes -o wide
```

Key Takeaways

- [x] Cordon, drain, upgrade, and validate — in that order.
- [x] Use PDBs to keep critical workloads available.
- [x] Upgrade nodes in a controlled sequence (not all at once).
- [x] Review autoscaler settings before and during upgrades.
- [x] Ensure workloads reschedule successfully and stay healthy.
- [x] Validate the cluster and monitor for issues after the upgrade.

Zero-Downtime Database Schema Migrations

Safely evolve your database schema in production without service interruption or data loss.

What It Covers

Techniques and practices to change a database schema in production while keeping the application available and stable.

Expand-and-Contract Pattern



- **Expand:** add new, backward-compatible schema (e.g., new column).
- **Contract:** remove old schema after all traffic uses the new version.

Backward-Compatible Changes

- Make schema changes that work with both old and new application versions.
- Examples: add nullable columns, add new tables, create indexes, avoid breaking changes.

Dual-Read and Dual-Write Risks

- **Dual-read:** read from both old and new schema (adds complexity).
- **Dual-write:** write to both schemas (risk of data drift, consistency issues, and higher latency).

Migration Sequencing (Example Flow)

1. **Expand** — Add new schema (backward-compatible).
2. **Deploy (v1 + v2)** — Release app that works with both old and new schema.
3. **Migrate Data** — Backfill or synchronize data if needed.
4. **Switch Traffic** — Move reads/writes to the new schema (dual-read/write if required).
5. **Contract** — Remove old schema after full adoption and validation.

Schema Rollback Limitations

- Rollbacks are not always simple (e.g., dropped columns lose data).
- Once data is migrated, reverting can be complex or impossible.
- Requires a planned rollback strategy (e.g., keep old schema until fully stable).

Key Takeaways

- ✅ Use the expand-and-contract pattern for safety.
- ✅ Keep schema changes backward compatible.
- ✅ Be cautious with dual-read and dual-write complexity.
- ✅ Follow a clear, step-by-step migration sequence.
- ✅ Understand rollback limits before starting.

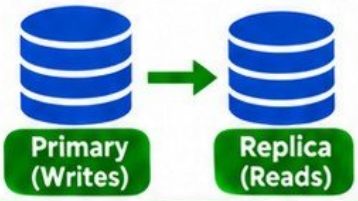
Database Maintenance, Failover, and Replication

Keep your databases highly available, consistent, and reliable during maintenance and unexpected failures.

What It Covers

Practical strategies for maintaining, replicating, and failing over databases with minimal or no downtime.

Primary and Replica Maintenance



- Maintain the replica first (e.g., patch, upgrade, test).
- Keep replication healthy before touching the primary.
- Use a controlled switchover for primary maintenance.

Replication Lag

- The delay between changes on the primary and the replica.
- Monitor lag continuously (e.g., seconds or bytes).
- Ensure lag is near zero before a failover or switchover.

Leader Promotion (Failover)



- Automatically or manually promote a replica to primary.
- Verify the new leader is healthy and accepting writes.
- Redirect application traffic to the new primary.

Connection Management

- Use a connection pool or proxy (e.g., PgBouncer, ProxySQL, RDS Proxy).
- Update connection endpoints automatically during failover.
- Set timeouts and retries to handle brief transitions.

Transaction Safety



- Use ACID-compliant transactions (Atomicity, Consistency, Isolation, Durability).
- Avoid long-running transactions during maintenance.
- Ensure writes are durable before confirming a failover or switchover.

Failover Validation (Checklist)

1. **Trigger** — Simulate or perform a controlled failover.
2. **Promote** — Confirm the replica is promoted to primary.
3. **Reconnect** — Verify applications reconnect successfully.
4. **Validate** — Check data consistency and replication health.
5. **Monitor** — Watch metrics (latency, errors, replication lag).

Common Risks

- High replication lag during maintenance.
- Data inconsistency from improper failover.
- Application connection failures after promotion.
- Long transactions blocking maintenance.
- Failover that is not fully validated.

Key Takeaways

- ✓ Maintain replicas first, then the primary.
- ✓ Monitor and control replication lag.
- ✓ Use automated, tested failover procedures.
- ✓ Ensure transaction safety and data consistency.
- ✓ Always validate failover and application health.

Safe Infrastructure Changes with Terraform

Make reliable and low-risk infrastructure changes using Terraform with proper review, control, and recovery practices.

What It Covers

Terraform

Safe and controlled changes to cloud infrastructure using Terraform while avoiding downtime, drift, or broken state.

Plan Review



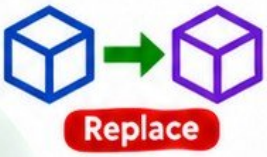
- Review the plan output before applying changes.
- Check what will be created, updated, or destroyed.
- Ensure changes are expected and low risk.

State Locking



- Prevent multiple users or pipelines from modifying the same state at once.
- Use remote state with locking (e.g., S3 + DynamoDB, Terraform Cloud/Enterprise).
- Avoid state corruption and conflicting changes.

Resource Replacement Detection



- Identify when a change will replace a resource (destroy and recreate).
- Check the plan for "-/+" (replace) actions.
- Assess impact and decide if a safer approach is needed.

Lifecycle Controls



- Use lifecycle settings to control how resources change.
- Examples: prevent_destroy, create_before_destroy, (gnore_changes.
- Reduce risk of accidental deletion and downtime.

Dependency Changes



- Understand and manage resource dependencies (e.g., VPC → subnets → instances).
- Make changes in the right order.
- Review how a change affects dependent resources.

Recovery from Partially Applied Changes

1. Detect
Identify a failed or partial apply (check output and remote state).

2. Assess
Compare state vs. actual resources to understand what changed.

3. Recover
Re-run apply, fix the issue, or manually reconcile the state.

4. Validate
Confirm resources are in the expected state and working correctly.

5. Prevent
Improve plans, add tests, and use locking to avoid recurrence.

Common Risks



- Accidentally replacing critical resources.
- State lock conflicts or stale locks.
- Partial applies that leave resources inconsistent.
- Unexpected changes to dependent resources.
- Downtime from unplanned resource recreation.

Key Takeaways

- Always review the plan before apply.
- Use remote state with locking.
- Watch for and manage resource replacements.
- Use lifecycle controls to reduce risk.
- Understand dependencies and change order.
- Have a clear recovery process for partial applies.

Load Balancer, DNS, and Network Maintenance

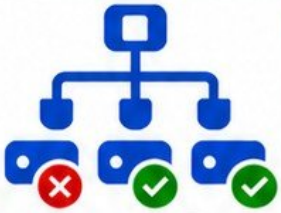
Perform infrastructure network changes safely and with minimal impact on users and running services.

What It Covers



Practical steps to maintain and update load balancers, DNS, and network settings without causing downtime or service disruption.

Connection Draining



- Stop sending new connections to a target before maintenance.
- Allow existing connections to finish (drain).
- Use a drain timeout (e.g., 30–300 seconds) based on application behavior.

DNS Time-to-Live (TTL)



- Use a lower TTL (e.g., 60–300 seconds) before planned changes.
- Allows faster propagation of new routes or IPs.
- Restore a higher TTL after the change for stability (e.g., 1 hour or more).

Routing Changes



- Update load balancer or DNS routing to the new target (e.g., new version, new region).
- Use weighted or staged routing for a safe transition.
- Validate traffic flow and application health after the change.

Firewall Updates



- Apply changes in a controlled way (e.g., allow new IPs first, then remove old ones).
- Review and test rules before and after the update.
- Keep a rollback plan to restore previous rules quickly.

Certificate Rotation



- Rotate TLS/SSL certificates before they expire.
- Use overlapping validity (old and new) to avoid downtime.
- Automate rotation where possible (e.g., cert-manager, ACME).

Network Cutover Verification (End-to-End)

1. Prepare



Confirm change details, TTL, and rollback plan.

2. Cutover



Update DNS/LB routing and apply network changes.

3. Verify



Check health, latency, error rate, and connectivity.

4. Monitor



Watch traffic and system metrics for issues.

5. Rollback (If Needed)



Quickly revert to the previous configuration.

Common Risks



- Active connections dropped too early.
- DNS changes taking longer than expected.
- Incorrect firewall rules blocking traffic.
- Expired or misconfigured certificates.
- Routing to an unhealthy or wrong target.

Key Takeaways

- ✓ Drain connections before removing or updating targets.
- ✓ Use an appropriate DNS TTL for faster, safer changes.
- ✓ Apply and test firewall updates in a controlled order.
- ✓ Rotate certificates before expiry and validate them.
- ✓ Verify the cutover end-to-end and have a rollback ready.

Secrets, Certificates, and Credential Rotation

Rotate secrets, certificates, and credentials safely and without service interruption using a controlled and staged approach.

What It Covers



Strategies and practices for securely rotating secrets, certificates, and credentials in production with minimal risk.

Overlapping Credential Validity



- Keep old and new credentials valid at the same time.
- Allow applications to switch gradually.
- Reduce the risk of outages during rotation.

Staged Rotation



- Rotate in controlled stages (e.g., dev → staging → prod).
- Update a small group of services first.
- Verify health and stability before full rollout.

Secret Distribution



- Store and manage secrets in a secure vault (e.g., AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).
- Distribute updates automatically to all dependent services.
- Enforce access control and audit usage.

Certificate Renewal



- Renew TLS/SSL certificates before they expire.
- Use automation (e.g., ACME, cert-manager) for renewal and deployment.
- Validate the new certificate (endpoints, trust chain, and routing) before retiring the old one.

Dependency Coordination



- Identify all services, APIs, and integrations that use the credential.
- Coordinate the rotation order (e.g., database, apps, CI/CD, jobs).
- Communicate changes and confirm compatibility across dependencies.

Credential Rotation Workflow (Controlled and Safe)

1. Prepare



Identify credentials, dependencies, and rotation plan.

2. Provision



Create the new secret/certificate with overlapping validity.

3. Deploy



Distribute and update services in stages.

4. Validate



Check health, logs, and integrations across all services.

5. Revoke



Remove the old credential after full confirmation.

Common Risks



- Rotating too quickly without overlap.
- Missing a dependency that still uses the old credential.
- Expired certificates causing outages.
- Inconsistent updates across environments.

Key Takeaways

- ✓ Always use overlapping validity for safe rotation.
- ✓ Rotate in stages and verify at each step.
- ✓ Manage secrets in a secure, centralized vault.
- ✓ Coordinate and test all dependent services.
- ✓ Revoke old credentials only after full validation.

Observability During Production Maintenance

Maintain visibility, control, and confidence during production changes using the right observability practices and validation checks.

What It Covers

How to use observability (metrics, logs, traces, and checks) to safely run and validate production maintenance.

Golden Signals

- The core indicators for service health:
- Latency: response time.
- Traffic: request volume.
- Errors: failure rate.
- Saturation: resource usage (CPU, memory, disk, etc.).

Baseline Comparisons

- Compare current metrics with a known good baseline (before the change).
- Define acceptable thresholds (e.g., ≤ 5% latency increase).
- Detect and investigate deviations early.

Synthetic Monitoring

- Run automated, scripted checks that mimic real users (e.g., login, API call, checkout).
- Validate key user journeys during and after maintenance.
- Provide an early signal if something breaks.

Distributed Tracing

- Trace requests across multiple services (e.g., with OpenTelemetry, Jaeger, Tempo).
- Identify latency, errors, and bottlenecks introduced by the change.
- Confirm end-to-end service health after the update.

Deployment Annotations

- Add clear annotations or labels to deployments (e.g., version, change ID, owner).
- Correlate metric and log changes with the exact release.
- Make it easier to analyze impact and roll back if needed.

Alert Management

- Adjust alerts during planned maintenance (e.g., suppress or lower non-critical alerts).
- Keep critical alerts active (e.g., availability, errors, data loss).
- Review and reset alert rules after the maintenance.

Maintenance Verification Dashboard

- Use a dedicated dashboard to track the change in real time.
- Include golden signals, key business metrics, and health checks.
- Verify service stability, performance, and error rates before closing the maintenance.

Common Risks

- Missing or incorrect baselines.
- Alert noise during the change.
- Undetected performance regressions.
- Broken user flows not covered by checks.
- Lack of traceability for the release.

Key Takeaways

- ✓ Use the golden signals to track service health.
- ✓ Always compare against a known baseline.
- ✓ Run synthetic checks for critical user journeys.
- ✓ Correlate changes with deployment annotations.
- ✓ Use a focused dashboard to verify a successful change.

Automated Maintenance Gates and Rollback Engineering

Use automated checks and controlled rollback to make production maintenance safe, fast, and low-risk.

What It Covers

Automation patterns and controls that decide when to proceed, pause, abort, or roll back a production change.

Predeployment Checks

- Validate code, configuration, and infrastructure changes.
- Run automated tests (lint, unit, integration, security).
- Check dependency and compatibility risks.
- Block the deployment if critical checks fail.

Health-Based Promotion

- Automatically promote a change when health metrics are within acceptable limits (e.g., error rate, latency, CPU).
- Use canary or progressive traffic with real-time checks.
- Require a stable period (e.g., 5–15 minutes) before full promotion.

Automated Aborts

- Automatically stop the rollout when critical metrics breach thresholds.
- Examples: high error rate, latency spike, or failed health checks.
- Revert traffic to the last stable version immediately.

Rollback Triggers

- Define clear, automated conditions that trigger a rollback (e.g., error rate > 2%, p95 latency > threshold).
- Keep a tested and versioned rollback path (artifacts, configs, database changes).
- Ensure rollback is fast, reliable, and repeatable.

Feature Flags

- Enable or disable features without a new deployment.
- Gradually roll out to a subset of users or cohorts.
- Quickly turn off a feature if issues are detected.
- Use flags for risky changes, experiments, or phased releases.

Recovery-Time Considerations

1. **Detect** — Identify an issue or metric breach quickly.
2. **Decide** — Confirm the rollback or abort action (automated).
3. **Recover** — Restore last known good version and configuration.
4. **Validate** — Verify service health, metrics, and user impact after recovery.
5. **Measure** — Track recovery time (RTO) and confirm stability (RPO impact).

Common Risks

- Incomplete or weak predeployment checks.
- False positives or incorrect abort triggers.
- Slow or untested rollback paths.
- Feature flags misconfigured or left enabled.
- Recovery time exceeding business targets.

Key Takeaways

- ✓ Automate checks, promotion, aborts, and rollbacks.
- ✓ Use clear, measurable health criteria for decisions.
- ✓ Leverage feature flags to reduce production risk.
- ✓ Test rollback regularly and keep it ready.
- ✓ Define and monitor recovery-time targets (RTO/RPO).

Failure Scenarios During Zero-Downtime Changes

Common failures that can occur during zero-downtime changes, their impact, and how to respond and recover quickly.

Stuck Rollouts



- A deployment stays in progress and never completes.
- Causes: failing health checks, misconfiguration, or insufficient resources.
- Fix: check rollout status, logs, and events; fix the root cause, then resume or roll back.

Unavailable Replicas



- New replicas fail to start or become ready.
- Causes: bad image, failed readiness probe, missing secrets/config, or resource limits.
- Fix: inspect pod logs and events, correct the issue, and redeploy or roll back.

Failed Node Drains



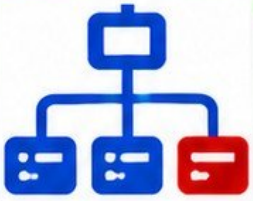
- A node cannot be drained (e.g., pods won't evict).
- Causes: PodDisruptionBudget (PDB) blocking, long-running pods, or missing eviction rules.
- Fix: review PDBs, allow controlled disruption, handle critical pods manually, then drain again.

Database Migration Failures



- A schema migration fails or leaves the database in a partial state.
- Causes: incompatible changes, long locks, or missing rollback plan.
- Fix: stop the rollout, restore from backup or previous schema, and rework the migration.

Traffic Imbalance



- Traffic is not evenly distributed across replicas or zones.
- Causes: incorrect load balancer config, unhealthy instances, or DNS/TTL issues.
- Fix: verify load balancer health, weights, and targets; refresh DNS if needed; restore balanced routing.

Cascading Dependency Failures



- A change in one service causes multiple dependent services to fail.
- Causes: incompatible API changes, config drift, or shared resource limits.
- Fix: isolate the change, roll back the triggering service, and restore dependencies in order.

Response and Recovery Flow

1. Detect
Identify the failure quickly (metrics, alerts, logs, events).

2. Diagnose
Find the root cause (e.g., rollout, node, DB, traffic, or dependency).

3. Mitigate
Take immediate action to stop further impact (e.g., pause, drain, or isolate).

4. Recover
Roll back or restore to a known good state.

5. Validate
Confirm service health, stability, and no residual issues.

Common Risks



- Incomplete or stuck deployments.
- Extended service unavailability.
- Data inconsistency or partial migrations.
- Wider outage from dependency failures.

Key Takeaways

- Expect and plan for failures, even with zero-downtime.
- Use health checks, PDBs, and safe rollback strategies.
- Monitor for early signs (errors, latency, replica health).
- Isolate changes and control blast radius.
- Run post-incident validation before closing the change.

Production Maintenance Incident Simulation

A realistic, end-to-end maintenance scenario across Kubernetes, databases, load balancing, and observability, from diagnosis to recovery and validation.

Scenario Overview

Perform a planned maintenance (Kubernetes node upgrade, database minor upgrade, and load balancer change) that encounters unexpected issues and requires incident response and recovery.

Production Environment

- Kubernentes: multi-node production cluster.
- Database: primary + replicas (e.g., PostgreSQL/MySQL).
- Load Balancer: L7 (e.g., NGINX/ AWS ALB).
- Observability: metrics, logs, traces, and alerts (e.g., Prometheus, Grafana, Loki).

Incident Symptoms (What Goes Wrong)

- After the node upgrade starts, new pods remain "Pending."
- Database writes become slow and replicas fall behind.
- Traffic becomes uneven across services (one zone overloaded).
- Alerts fire: high latency, error rate increase, and replica lag.

Investigation and Diagnosis

- Check Kubernetes: pod status, node health, and scheduling events.
- Check database: replication lag, locks, and resource usage.
- Check load balancer: target health and traffic distribution.
- Use observability: correlate metrics, logs, and traces to find the root cause.

Decision and Recovery Actions

1. Assess — Confirm impact, scope, and blast radius.

2. Stabilize — Pause the rollout, restore capacity (e.g., add nodes or scale up).

3. Fix — Resolve the root cause (e.g., scheduling constraints, DB resource limits, or LB misconfig).

4. Recover — Resume the change safely (e.g., complete node upgrade and DB upgrade in controlled steps).

5. Validate — Confirm service health, performance baselines, and no user impact.

Post-Change Validation

- Verify service availability, latency, error rate, and throughput.
- Confirm database replication is healthy (lag back to normal).
- Ensure traffic is evenly balanced across all targets.
- Run synthetic checks and key user journeys.

Key Learning Points

- Plan maintenance with clear dependencies and rollback path.
- Monitor in real time and use data to decide.
- Pause and recover safely when signals degrade.
- Validate not just "up" status, but also performance and business health.

Common Pitfalls

- Proceeding despite early warning signals.
- Ignoring database replication lag.
- Relying on a single health check.
- Not validating traffic balance after changes.

Key Takeaways

- ✓ Use observability to drive every decision.
- ✓ Treat maintenance like a real incident with a plan.
- ✓ Stabilize, fix, recover, and validate in stages.
- ✓ Always finish with a post-change review and improvements.

Senior Engineer Production Maintenance Assessment

A comprehensive, real-world assessment to test your ability to plan, execute, troubleshoot, and validate production maintenance safely and effectively.

Assessment Focus Areas

- Production change planning.
- Troubleshooting and decision-making.
- Rollback and recovery strategy.
- Availability and risk trade-offs.
- End-to-end production readiness.

Assessment Format

- Scenario-based questions.
- Change-plan review and analysis.
- Troubleshooting decisions.
- Rollback reasoning.
- Practical production-readiness assessment.

Sample Scenario Questions

- A rollout is stuck at 50%. What are the likely causes and next steps?
- A database migration is running slow and affecting latency. What do you do?
- Traffic is imbalanced after a deployment. How do you diagnose and fix it?

Change-Plan Review (What to Check)

- Clear objective and success criteria.
- Risk assessment and blast radius.
- Rollback and recovery plan.
- Monitoring, alerts, and validation.
- Communication and change window.
- Dependency and sequencing.

Availability Trade-Offs



- When to prefer faster delivery vs. higher availability.
- Impact of rolling vs. blue-green vs. canary deployments.
- Acceptable risk for schema changes, configuration updates, and dependency upgrades.

Rollback Reasoning

- When to roll back immediately vs. when to stabilize and monitor.
- How to decide using metrics (error rate, latency, saturation).
- Handling partial changes and data migrations.
- Ensuring a safe and repeatable rollback process.

Practical Production-Readiness Assessment

1. Plan	2. Execute	3. Monitor	4. Troubleshoot	5. Validate
Review the scenario, define the change plan, risks, and rollback.	Perform the change in a controlled, safe manner (simulated).	Validate health using metrics, logs, and traces.	Diagnose and resolve an injected issue (e.g., rollout, DB, or traffic issue).	Confirm stability, availability, and successful change completion.

Evaluation Criteria

- Quality and completeness of the change plan.
- Accuracy and speed of troubleshooting.
- Correct use of rollback and recovery options.
- Ability to manage risk and maintain availability.
- Overall production-readiness and communication.

Key Takeaways

- Plan for failure, not just success.
- Use data and observability to make decisions.
- Choose the right trade-off for business needs.
- Keep rollbacks simple, tested, and ready.
- A successful change is validated, not assumed.